

Exercícios — Aula 17 — Introdução ao backtracking

Técnicas de Programação - 2026/02

Última atualização: July 31, 2026

Instruções

- Leia o pseudocódigo antes de executar mentalmente.
- Em backtracking, acompanhe estado parcial, índice atual, escolha feita e desfazer.
- Quando uma solução for registrada, pense se é necessário copiar o estado parcial.

Exercício 1 — Simulação de subconjuntos

Simule o algoritmo para $v = [1, 2, 3, 4]$.

Algorithm 1: GerarSubconjuntos

Result: Executa GERAR-SUBCONJUNTOS.

Input : array v

Output: list respostas

```
respostas ← LISTA-VAZIA()
atual ← LISTA-VAZIA()
BACKTRACK( $v, 0, atual, respostas$ )
return respostas
```

Algorithm 2: Backtrack

Result: Executa BACKTRACK.

Input : array v , int i , list atual, list respostas

Output: none

```
if  $i = TAMANHO(v)$  then
    ADICIONAR-COPIA( $respostas, atual$ )
    return
end
BACKTRACK( $v, i + 1, atual, respostas$ )
ADICIONAR( $atual, v[i]$ )
BACKTRACK( $v, i + 1, atual, respostas$ )
REMOVER-ULTIMO( $atual$ )
```

Preencha a árvore de decisões. Use N para “não incluir” e S para “incluir”. Exemplo:

```
i=0, atual=[]
|-- N 1 -> i=1, atual=[]
|   |-- N 2 -> ...
|   `-- S 2 -> ...
`-- S 1 -> i=1, atual=[1]
     |-- N 2 -> ...
     `-- S 2 -> ...
```

Agora liste, na ordem em que o algoritmo registra, todos os subconjuntos adicionados a **respostas**.

Exercício 2 — Estado, candidatos e caso base

Para o algoritmo do Exercício 1, complete a tabela.

Pergunta	Resposta
Qual é o estado parcial?	
Qual variável indica a decisão atual?	
Quais são os candidatos em cada nível?	
Qual é o caso base?	
O que é registrado como resposta?	
Qual operação desfaz uma escolha?	

Depois responda:

1. Por que cada elemento gera duas possibilidades?
2. O algoritmo gera subconjuntos repetidos?
3. A ordem dos subconjuntos registrados muda se a chamada para “incluir” vier antes da chamada para “não incluir”?

Exercício 3 — Problema de backtracking com erro

O pseudocódigo abaixo apresenta um problema.

Algorithm 3: BacktrackComErro

Result: Executa BACKTRACK-COM-ERRO.**Input** : array v , int i , list $atual$, list $respostas$ **Output:** none

```
if  $i = TAMANHO(v)$  then
    ADICIONAR-COPIA( $respostas, atual$ )
    return
end
BACKTRACK-COM-ERRO( $v, i + 1, atual, respostas$ )
ADICIONAR( $atual, v[i]$ )
BACKTRACK-COM-ERRO( $v, i + 1, atual, respostas$ )
```

1. Simule para $v = [1, 2, 3]$.
2. Quais respostas são registradas?
3. Qual elemento fica indevidamente em **atual** ao voltar para outro ramo?
4. Como corrigir o problema?

Exercício 4 — Falta de cópia

O pseudocódigo abaixo apresenta outro problema.

Algorithm 4: BacktrackSemCopia

Result: Executa BACKTRACK.

Input : array v , int i , list $atual$, list $respostas$

Output: none

```
if  $i = TAMANHO(v)$  then
    ADICIONAR( $respostas, atual$ )
    return
end
BACKTRACK( $v, i + 1, atual, respostas$ )
ADICIONAR( $atual, v[i]$ )
BACKTRACK( $v, i + 1, atual, respostas$ )
REMOVER-ULTIMO( $atual$ )
```

Implemente o pseudocódigo acima em Java e responda:

1. Quais respostas são registradas?
2. Por que **atual** continua mudando depois de ser adicionada a **respostas**?
3. Qual linha deve ser trocada para evitar o problema?
4. Escreva a versão corrigida do caso base.

Exercício 5 — Subconjuntos com soma par

Escreva pseudocódigo para gerar apenas subconjuntos cuja soma é par.

Contrato:

- entrada: array `v` de inteiros;
- saída: lista de subconjuntos com soma par;
- use backtracking com decisões incluir/não incluir;
- carregue `soma_atual` como parte do estado;

Teste com:

- `v = [1, 2, 3]`;
- `v = [2, 4]`;
- `v = [1]`.

Exercício 6 — Número de folhas

Considere o algoritmo do Exercício 1. Complete a tabela para n elementos.

n	escolhas por elemento	número de folhas	número de subconjuntos
0			
1			
2			
3			
4			
5			

Depois responda:

1. Qual a relação entre número de elementos e o número de subconjuntos?
2. Qual a relação entre número de elementos e o número de folhas?
3. Qual é a profundidade máxima da recursão?